

Capa de transporte

Transmisión fiable con TCP

Programación y administración de redes

Grado en *Ingeniería Informática*

Departamento de Informática. Universidad de Jaén

Objetivos

General

Conocer los mecanismos que el protocolo TCP emplea para garantizar una comunicación fiable entre dos host

Específicos

- Determinar cómo se usan los números de secuencia y reconocimiento para hacer la **entrega en orden** y detectar pérdidas de datos
- Conocer el algoritmo básico de **reenvío de paquetes** perdidos
- Analizar las **situaciones** que llevan a TCP a reenviar segmentos
- Identificar los elementos de control y **temporización** de reenvíos
- Saber cómo se usa el método de **reenvío rápido** de segmentos para facilitar la recuperación de datos perdidos

Transmisión fiable con el protocolo TCP



Transmisión fiable con TCP

Responsabilidades

TCP ofrece a las aplicaciones una abstracción de flujo de datos fiable, lo que le hace un protocolo considerablemente complejo al ser responsable de dividir los datos en porciones, garantizar recepción en orden, control de flujo y de congestión, etc.

RFC relacionados con TCP

Existen decenas de RFC relacionados con el funcionamiento de TCP, un protocolo que ha evolucionado notablemente en las últimas décadas. Algunos de los fundamentales son los enumerados en la tabla inferior.

RFC	Contenido	RFC relacionados
9293	Descripción general de TCP	793, 879, 2873, 6093, 6429, 6528, 6691
6298	Gestión del temporizador de retransmisión	2988, 1122, 5682
6675	Mecanismo de SACK básico	3517, 2018, 2883
5681	Algoritmos de control de congestión	3782, 6582

En el [RFC 7414: A Roadmap for TCP Specification Documents](#) se facilita una guía sobre los RFC que deberían consultarse para cada aspecto relacionado con el funcionamiento del protocolo TCP.

Transmisión fiable con TCP

Aspectos fundamentales

En este tema nos centramos en el estudio de los aspectos fundamentales tras la transmisión fiable de datos ofrecida por TCP, como son:

- La gestión de los números de secuencia y de reconocimiento, esenciales para garantizar la **recepción en orden** y detectar **paquetes perdidos**
- El funcionamiento básico del temporizador de retransmisión, herramienta que el emisor emplea para saber **cuándo ha de reenviar** segmentos y reducir la tasa de envío
- Los fundamentos de otros mecanismos de reenvío, como la retransmisión rápida por **ACK duplicado** o el **ACK selectivo** (SACK)

Algunos aspectos se han simplificado para facilitar la comprensión general, pero siempre es posible consultar el RFC correspondiente para conocer todos los detalles al respecto.

Números de secuencia y de reconocimiento

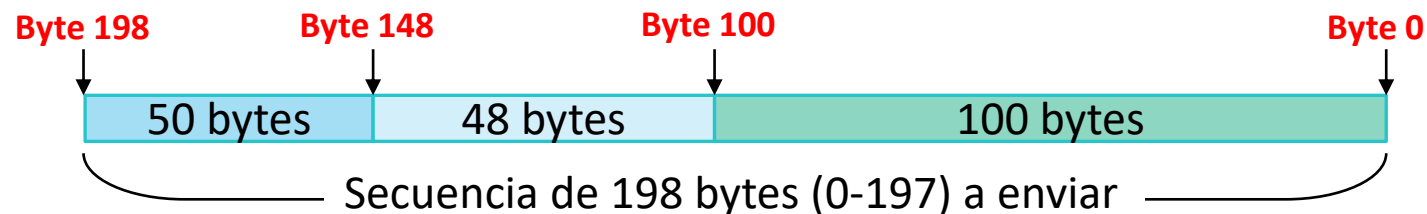


La abstracción del flujo de datos

Funcionamiento

Establecida la conexión, TCP ofrece a las aplicaciones una abstracción que les permite operar sobre un flujo de datos continuo, sin tener que preocuparse por empaquetar los datos en segmentos de tamaño fijo

- La **aplicación emisora** escribe en el *socket* como lo haría en un archivo de texto corriente:
 - Los datos enviados a través del *socket* se almacenan en el **buffer de envío** TCP, situado en la capa de transporte
 - Múltiples escrituras desde la aplicación pueden **concatenarse en el buffer** antes de que se envíe el segmento
 - Los segmentos se envían cuando **están llenos** o transcurrido un **cierto tiempo** aunque no lo estén
- La **aplicación receptora** lee datos del *socket* como lo haría de un archivo de texto:
 - El *host* almacena los datos de los segmentos recibidos en el **buffer de recepción** TCP, en la capa de transporte
 - TCP puede **almacenar varios segmentos** mientras que la aplicación lee datos del buffer según lo necesita
 - El buffer de recepción garantiza que la aplicación **recibirá los datos en el orden** en que se enviaron

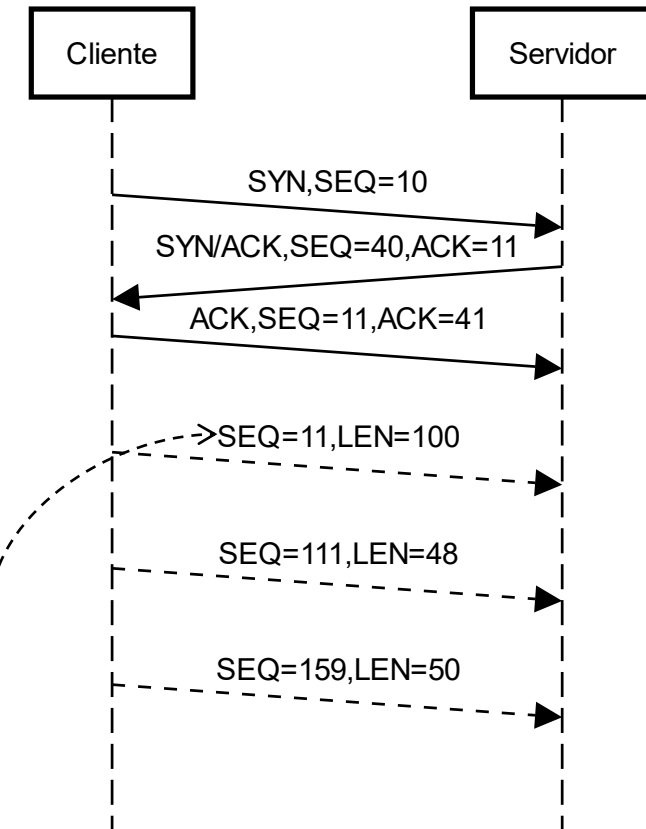
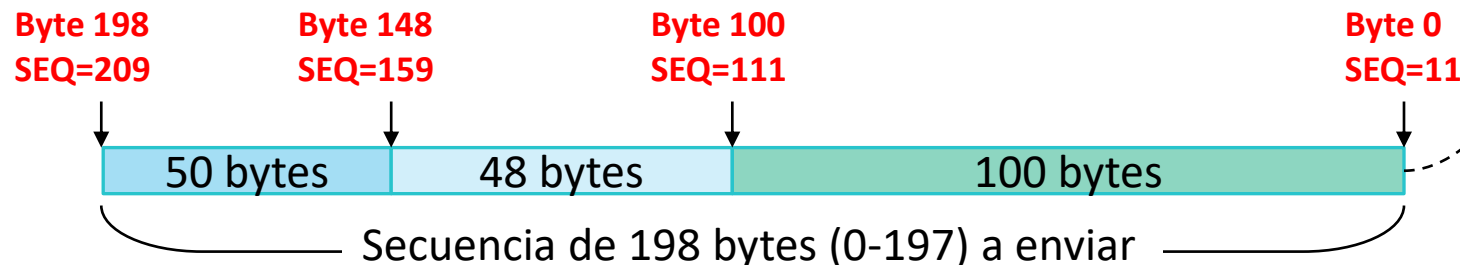


Números de secuencia

Funcionamiento

Cada uno de los bytes del flujo continuo de datos que un host envía a otro durante una conexión TCP tiene asignada una posición única a la que se denomina número de secuencia

- **Inicialización** del número de secuencia – Campo SEQ de la cabecera TCP:
 - Se inicializa durante el establecimiento de conexión TCP
 - Cada *host* comunica al otro cuál es su **valor inicial** para este campo
 - No se parte de 0 o 1, sino que se toma un valor **aleatorio** como número inicial
- **Actualización** del número de secuencia:
 - El *host* que emite indica el **número de byte**, dentro de toda la secuencia de bytes que se van a enviar, que corresponde al primero de los bytes del campo de datos que se está enviando
 - A continuación se **suma al campo SEQ** el número de bytes transmitidos, siendo ese el número de secuencia a incluir en el siguiente segmento
 - El proceso se repite hasta finalizar el envío de todo el flujo de datos y **cerrar la conexión**, mientras tanto el número de secuencia es actualizado en cada envío



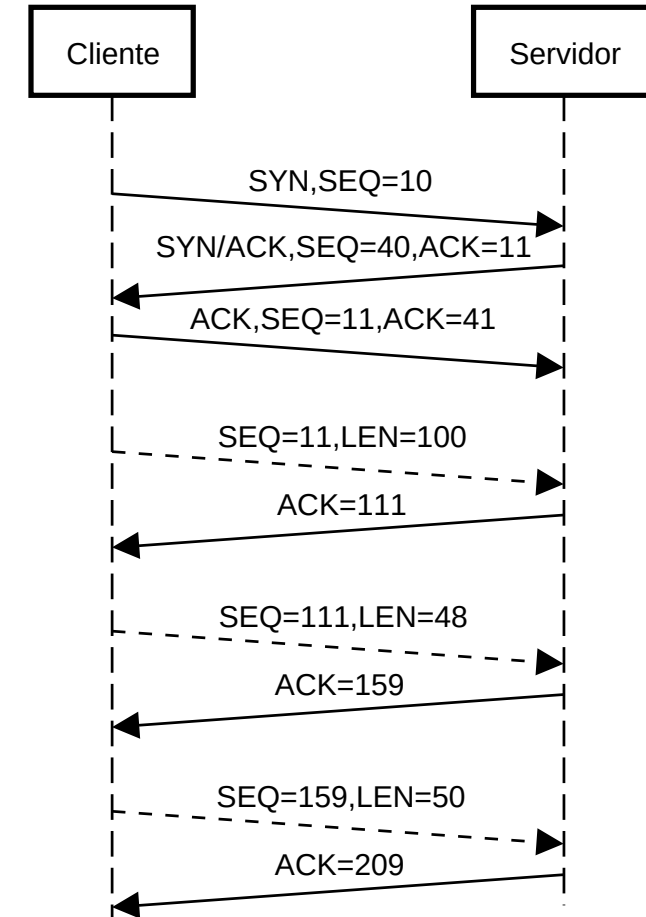
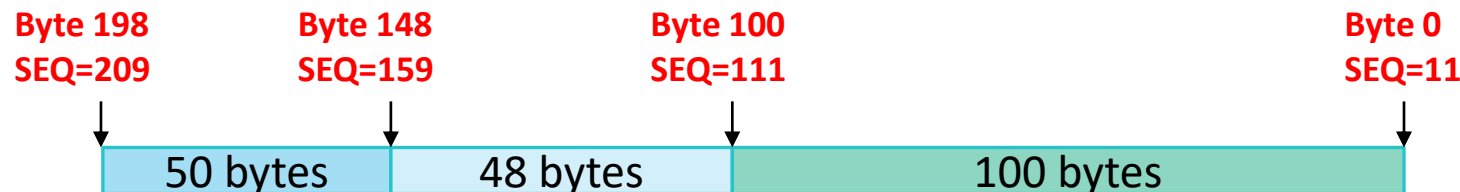
Números de reconocimiento

Funcionamiento

Para asegurar que no se pierden datos, el host emisor no elimina de su buffer los segmentos enviados hasta que el destinatario le confirma que los ha recibido

- **El campo ACK:**

- El **valor inicial** que un *host* da a este campo se establece durante el proceso de conexión, como ocurre con el número de secuencia
- El **nuevo valor** corresponderá al siguiente byte a recibir en el flujo
- El campo **ACK** solo es leído cuando **se activa el bit ACK** del campo **Flags** de la cabecera
- Un mismo segmento servirá para enviar datos al otro extremo de la conexión y, al tiempo, reconocer datos recibidos, mediante la **técnica de superposición o piggybacking**
- No es imprescindible que un *host* envíe un segmento de reconocimiento por cada segmento de datos recibido, se puede usar el *delayed ACK* (**reconocimiento acumulativo**) sobre el que profundizaremos después
- Al recibir un **ACK**, el *host* emisor ya puede eliminar de su buffer los segmentos cuyo número de **secuencia sea menor** que el del **ACK**



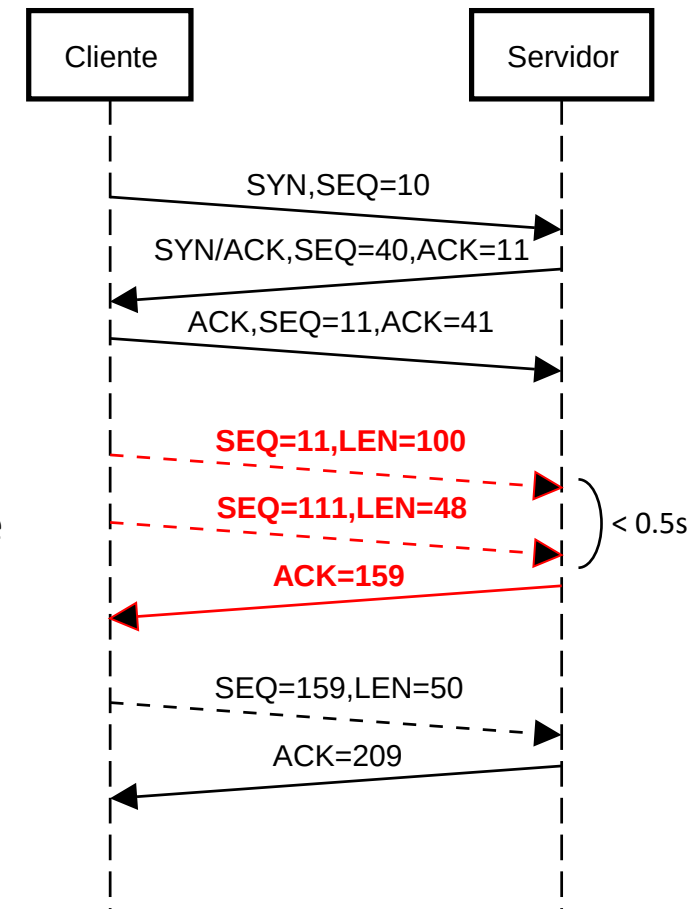
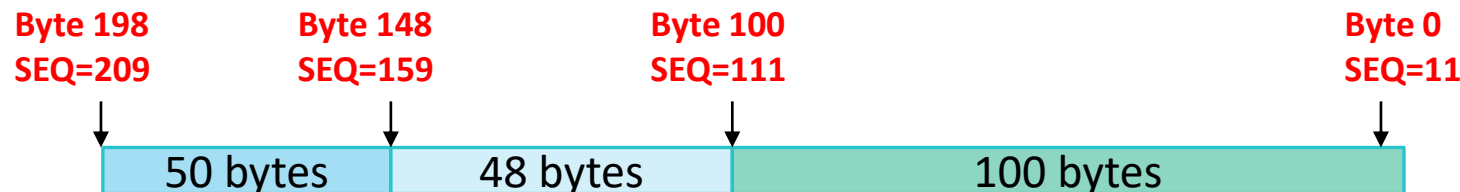
Reconocimiento acumulado

Descripción

El uso de un solo ACK para reconocer la recepción de múltiples segmentos reduce el tráfico de la red, lo que ayuda a evitar la congestión de la misma. Esta técnica es conocida como Delayed ACK

- **Funcionamiento:**

- Cuando al *host* receptor le llega un segmento, no se envía el **ACK** de manera inmediata sino que pone en marcha un **temporizador de espera**
- En la mayoría de implementaciones TCP/IP, la espera máxima de dicho temporizador es de 0.5 segundos
- Si en ese lapso de tiempo se recibiesen segmentos adicionales, se enviará un único segmento de **ACK** reconociendo hasta el último byte recibido
- El envío de **ACK** por parte del receptor se producirá de manera inmediata en caso de que se reciban **dos segmentos completos**, llenos de datos (LEN=MSS), sin que expire el temporizador



El temporizador de retransmisión



Finalidad y casuísticas

Finalidad

El protocolo TCP ofrece un servicio fiable de transmisión de datos, por lo que en caso de pérdidas o errores será preciso que el host emisor retransmita al destinatario los segmentos afectados por el fallo

- Qué desencadena el reenvío de segmentos:
 - La no recepción de los reconocimientos por parte del receptor: el segmento no ha llegado o tiene errores
 - La recepción de segmentos con valores repetidos en el campo **ACK**
 - La recepción de un segmento **ACK** que incluya reconocimientos selectivos (**SACK**)
- Componentes precisos en el emisor para controlar estas casuísticas:
 - Un **temporizador de retransmisión**, cuya cuenta atrás señalará el momento en que sería preciso el reenvío de segmentos no reconocidos
 - Un valor de referencia, conocido como **RTO** (*Retransmission TimeOut*), que permita fijar el **lapso de tiempo** de dicho temporizador
 - Un algoritmo pensado para **actualizar el RTO** según cambien las condiciones de la red, de forma que no se reenvíen segmentos demasiado pronto o demasiado tarde

Inicialización del temporizador de retransmisión

Definición

El temporizador de retransmisión (TR) de TCP se inicia con el RTO actual. El valor inicial del RTO ha cambiado a lo largo del tiempo

Inicialización del RTO

- El valor inicial para RTO **se fija antes que establecerse la conexión** entre los *host*, de forma que afecta al tiempo que se esperará la respuesta a esa petición de conexión
- Originalmente se asignaba un valor de **3 segundos** al RTO a fin de evitar reenviar la solicitud de conexión demasiado pronto en caso de transitar por redes lentas
- En la actualidad el valor inicial del RTO se establece en **1 segundo**, según se establece en la sección 2 del [RFC 6298: Computing TCP's Retransmission Timer](#)
- El valor del RTO se actualizará en cuanto se tenga la primera **medida de RTT**, al recibir la respuesta a la petición de conexión

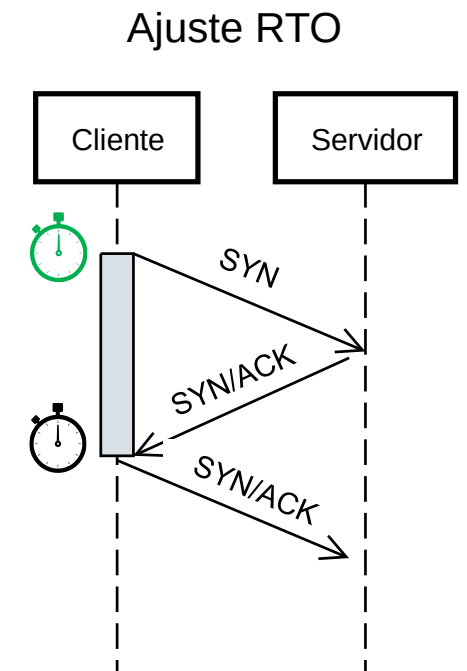
Ajuste normal del RTO

Definición

El RTO se ajusta constantemente, con cada respuesta recibida de la otra parte, mediante un algoritmo que trata de optimizar el uso de la red

Cálculo del RTO

- El valor inicial para RTO se fija antes de la conexión a **1 segundo**
- Cada extremo de la comunicación, **Cliente** y **Servidor** en el diagrama de la derecha, fijan su **RTO = RTT inicial**
- A lo largo del tiempo de conexión, se **muestra el tiempo de RTT** y se ajusta el RTO mediante una fórmula que tiene en cuenta valores previos del RTO y su varianza
- Asumiremos el uso de una fórmula simplificada, que era la usada en versiones anteriores de TCP: $RTO_t = \alpha RTO_{t-1} + (1 - \alpha)RTT$, siendo α un factor de suavizado cuyo valor recomendado es 0.9
- Siempre que no haya pérdida de segmentos, el RTO irá ajustándose de forma dinámica a las condiciones de la red en cada momento. Lo habitual es que el valor inicial de 1 segundo **se reduzca paulatinamente**, pudiendo llegar a 0.2 segundos (según implementación).



→ Se inicia el TR



→ Se detiene el TR sin que haya expirado



→ El TR expira

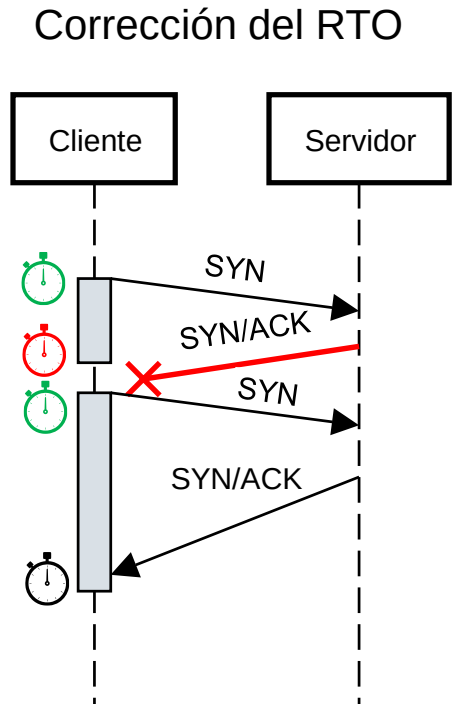
Ajuste del RTO ante expiración del TR

Definición

En caso de que el temporizador del emisor expire sin haber recibido la confirmación de recepción, el RTO se corrige duplicando su valor

Corrección del RTO

- En caso de que el temporizador expire, TCP aplica la técnica denominada **exponential backoff** con el fin de **reducir la congestión** en la red, que provocaría el reenvío de un mismo paquete múltiples veces con el mismo RTO
- **El RTO duplica** su valor antes de llevar a cabo la retransmisión del segmento cuya pérdida se ha detectado. El cálculo del RTO suavizado se deja en suspenso hasta en tanto no se reciban asentimientos de recepción desde la otra parte
- El valor máximo del RTO, en caso de que la pérdida de los segmentos enviados se repita, **alcanzará los 60 segundos**
- Los reenvíos se repiten hasta alcanzar el **límite de intentos de reenvío**, momento en que se cierra la conexión



Gestión del temporizador de retransmisión

Reglas básicas

El temporizador de retransmisión se gestiona con un conjunto de reglas sencillas:

- Al enviar un segmento, tanto si es retransmisión como si es un primer envío, **si el TR no estaba en marcha** se inicia con el RTO que corresponda, de forma que expire tras RTO segundos
- Cada vez que se recibe un ACK, si todavía hay datos en tránsito (segmentos pendientes de reconocer) se **reinicia el TR** tras ajustar el RTO según la fórmula ya descrita
- Cuando se reciba el ACK del último segmento de datos enviado, y por tanto no quedan segmentos por reconocer, el **TR se detiene**
- Si el TR expira, el **RTO se duplica**, se reenvía el segmento más antiguo sin reconocer y se reinicia el TR con el nuevo RTO
- La recepción de ACK repetidos o de SACK **no afecta al RTO**

Retransmisión de segmentos



Reglas de envío y reconocimiento de segmentos

Reglas generales

Al estudiar los casos de retransmisión descritos en las siguientes diapositivas se tendrán presentes las siguientes reglas generales de envío y reconocimiento de segmentos

- **Reglas generales de envío de segmentos:**

- El emisor enviará un segmento en cuanto los datos almacenados en el *buffer* de emisión alcancen el MSS
- Si el *buffer* contiene **menos de MSS bytes**, la emisión puede demorarse un tiempo a la espera de más datos
- Si el *buffer* contiene **más de MSS bytes**, el emisor puede enviar varios segmentos sin tiempo de espera
 - El número de segmentos que puede enviarse dependerá del espacio libre en el *buffer* de recepción de la otra parte
 - Habitualmente el número de segmentos que es posible enviar crece de forma rápida hasta detectar congestión

- **Reglas generales de reconocimiento de segmentos:**

- El receptor tiene un temporizador que se pone en marcha al recibir un segmento que es preciso reconocer y cuyo número de secuencia (**campo SEQ**) corresponde con el esperado
- Si se reciben dos segmentos seguidos llenos, con MSS bytes cada uno, se envía el **ACK de inmediato** y el temporizador se detiene
- Expirado el temporizador, se envía el **ACK acumulado** de todos los segmentos recibidos en el intervalo de tiempo
 - Podrían haberse recibido **más de dos segmentos** si su contenido (*payload*) no alcanza los MSS bytes
- El receptor enviará un **ACK inmediato** en caso de recibir un segmento con un número de secuencia inesperado

Retransmisión por expiración del TR

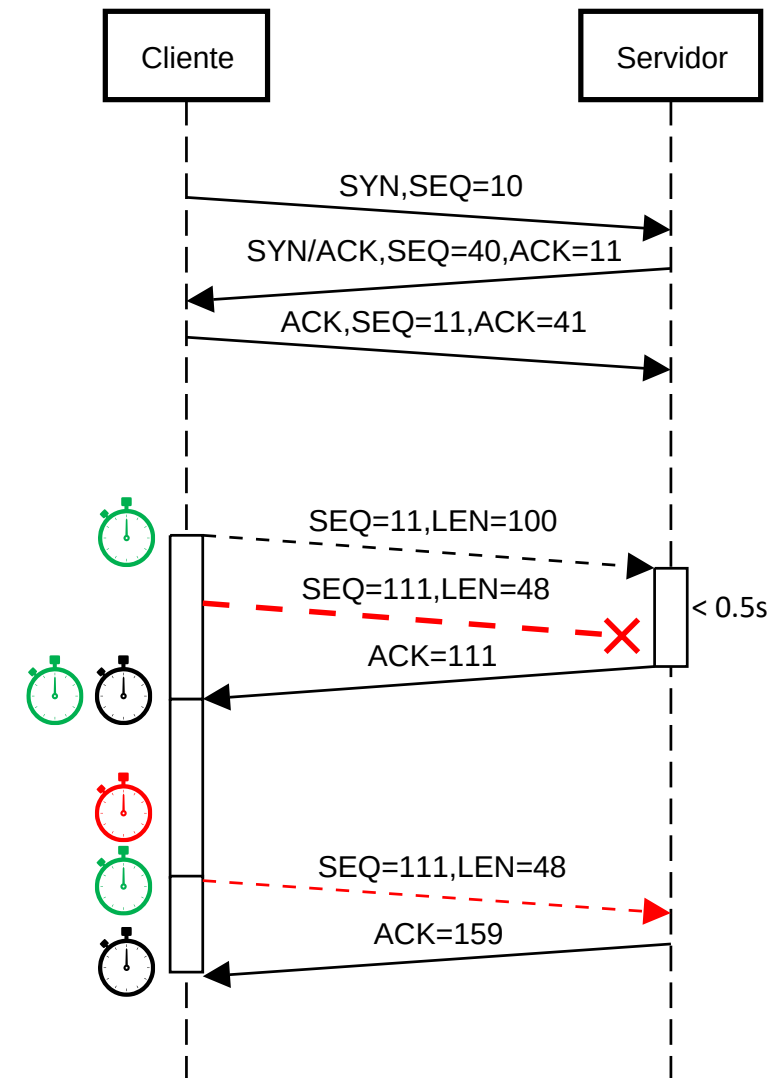
Descripción

El emisor envía un segmento y el temporizador asociado, ajustado según el RTO, expira antes de que se reciba el correspondiente ACK por parte del otro host

- Cuando un *host* envía un segmento mediante TCP:
 - Se agrega dicho segmento a una **cola de segmentos pendientes (CSP)** de reconocer
 - Si el TR no está en marcha, se inicia con tiempo = RTO
- En caso de recibirse un ACK antes de que el TR expire:
 - El segmento reconocido y todos los anteriores se eliminan de la cola
 - Si la cola no está vacía, se reinicia el TR con RTO, en caso contrario se detiene
- En caso de que expire el TR:
 - Se toma de la CSP el segmento con menor número de secuencia (el más antiguo) y se reenvía
 - Se ajusta el TR aplicando *exponential backoff*, de forma que $RTO = 2 \times RTO$

Cola de retransmisión de segmentos

50 bytes	SEQ=159	48 bytes	SEQ=111	100 bytes	SEQ=11
----------	---------	----------	---------	-----------	--------

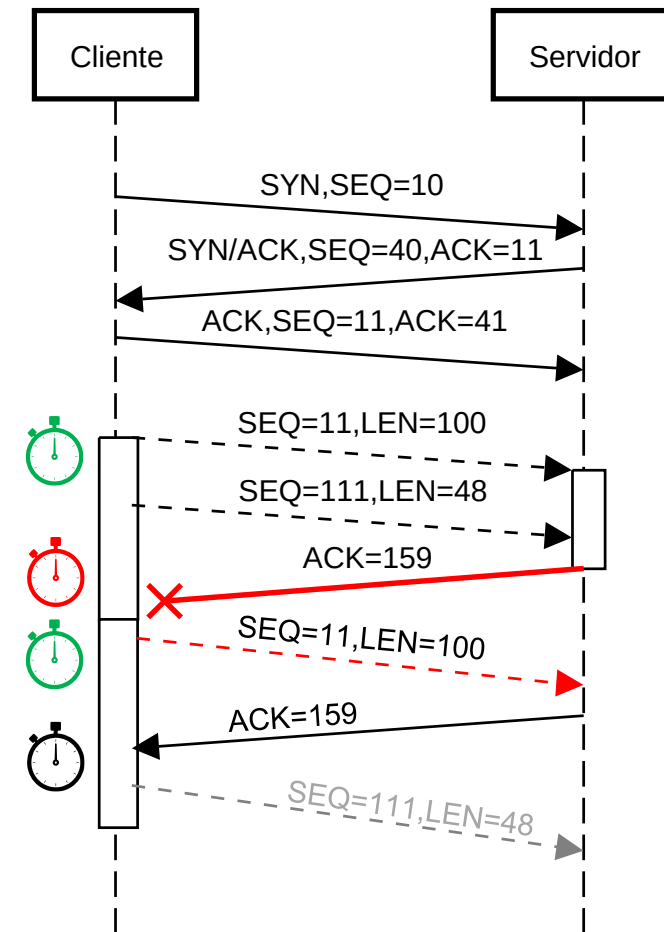


Retransmisión por pérdida del ACK

Descripción

La situación es similar, ya que el TR terminará expirando al no recibirse el ACK, pero las consecuencias pueden diferir según el número de segmentos alojados en la CSP

- **Envío de segmentos por parte de un *host*:**
 - Puede enviar tantos segmentos completos como permita el **buffer de recepción** de la otra parte, cuyo estado se informa en cada segmento recibido
 - En este ejemplo el **Cliente** envía dos segmentos al **Servidor**
- **Pérdida del ACK enviado por la otra parte:**
 - El *host* **Servidor** envía un **ACK** acumulado por los dos segmentos, pero se pierde
 - El TR del **Cliente** expira, por lo que se asume que ambos segmentos se han perdido
- **Retransmisión:**
 - El **Cliente** retransmite el primer segmento, que el **Servidor** ya tenía almacenado
 - El **Servidor** reconoce los dos segmentos que recibió, no solo el que acaba de llegar
 - Según temporización, el **Cliente** podría no llegar a enviar el segundo segmento, si le llega el **ACK** acumulado a tiempo, o hacerlo si el **ACK** se demora



Retransmisión rápida por ACK repetido

Descripción

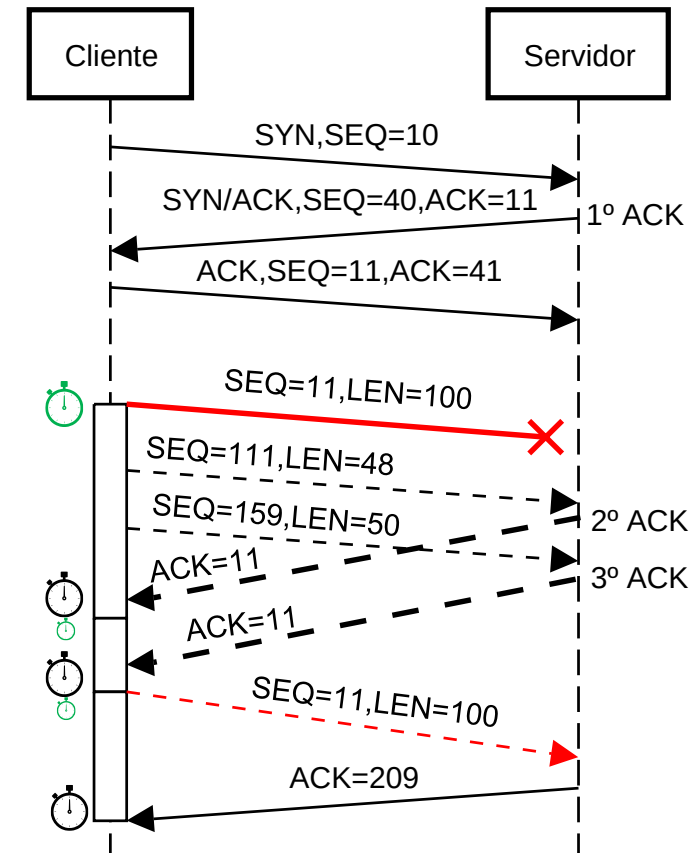
Los segmentos pueden llegar al destino desordenados, al seguir distintas rutas, así como perderse alguno sin que los otros se vean afectados. El host receptor informará al emisor de esta casuística mediante el envío de un ACK repetido

• Cuándo se envía un ACK repetido:

- Siempre que al receptor le llega un segmento con un número de secuencia inesperado, posterior al que debería, se envía **de forma inmediata** un **ACK** con el número de secuencia que se esperaba
- En ocasiones los segmentos llegan fuera de orden, por lo que el emisor **no envía de inmediato el segmento** que esperaba el receptor ya que es posible que le llegue antes de que expire el TR asociado

• Retransmisión rápida de segmentos perdido – al recibir tres ACK repetidos:

- Con cada recepción del **ACK** repetido, el emisor reinicia el TR pero no ajusta el RTO, ya que ese **ACK** le hace saber que los segmentos, aunque fuera de orden, están llegando al receptor
- Al recibir el tercer **ACK** repetido se activa el modo de **recuperación rápida** y se procede a enviar el segmento cuyo número de secuencia está solicitando el servidor
- Se pone en marcha el TR y se espera el siguiente **ACK**. Cuando este llega, según su valor:
 - Si el ACK **reconoce hasta el último segmento** enviado, como en el ejemplo de la derecha, se detiene el TR y el proceso finaliza
 - Si el ACK es un **reconocimiento parcial** hasta otro segmento perdido, el cliente lo envía de inmediato y repite el proceso hasta confirmar la recepción de todos los segmentos



ACK selectivo

Descripción

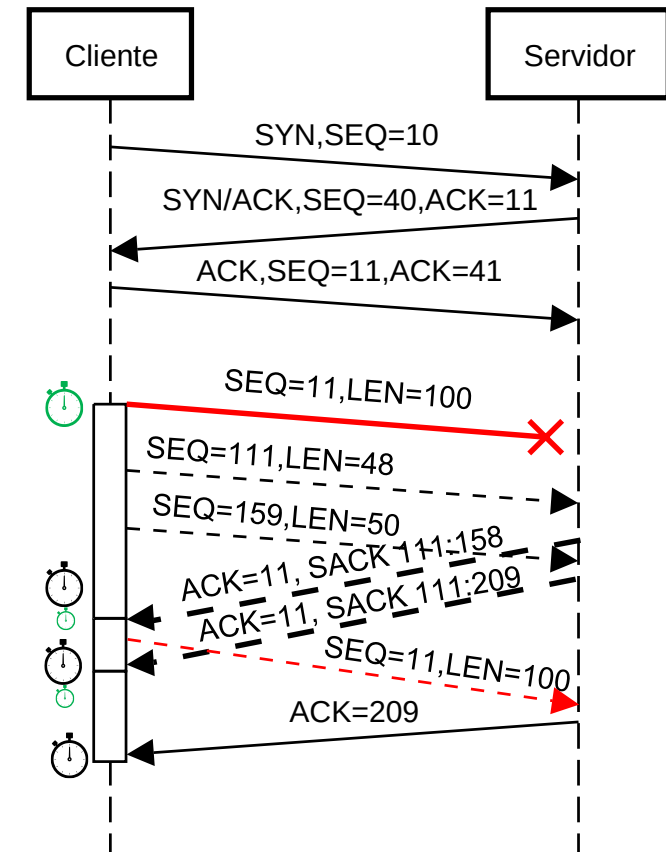
El host receptor al que le llegan segmentos fuera de orden, con un número de secuencia adelantado al que espera, puede informar al emisor de las partes recibidas mediante SACK

- **Definición de SACK:**

- El *Selective ACK* es una de las opciones que pueden incluirse en la cabecera de TCP y su finalidad es reducir el número de segmentos retransmitidos de manera innecesaria
- La opción **SACK** contiene uno o más intervalos de número de secuencia, con los que el receptor informa al emisor de las partes ya recibidas
- Cada intervalo está compuesto por dos números de 32 bits que indican el número de secuencia del **primer y último byte** de un segmento recibido

- **Funcionamiento:**

- Si el emisor recibe un segmento **ACK** con la opción **SACK** activa, no se espera a recibir tres **ACK** repetidos
- Se usan los intervalos indicados por **SACK** para determinar qué segmentos es preciso reenviar y hacerlo lo antes posible
- El **número de segmentos** perdidos que se reenvían de forma simultánea vendrá dado por un algoritmo que tiene en cuenta múltiples aspectos
 - El cliente sabe, al recibir un ACK, que los segmentos enviados están llegando al servidor, porque de lo contrario este no respondería, razón por la que **no se precipita en el envío** de segmentos perdidos ya que estos podrían encontrarse en tránsito



Actividades y recursos adicionales



Ejercicio para la sesión del 17 de marzo



Por cada ejercicio dibuja el cronograma correspondiente y responde a las cuestiones

Ejercicio 1:

- Se establece conexión entre dos *host*. **A** informa que su número de secuencia inicial (**ISN**) es 1000, y **B** que el suyo es 300.
- **A** envía un segmento TCP con 100 bytes de datos a **B**.
- ¿Cuál es el número de secuencia del segmento enviado por **A**?
- ¿Cuál es el número de ACK que enviará **B** al recibir este segmento?
- ¿Qué número de secuencia tendrá el siguiente segmento que **A** enviará, suponiendo que se envían 200 bytes de datos adicionales?

Ejercicio 2:

- Repite el ejercicio anterior, pero ahora asume que el segmento inicial de **A** contiene 500 bytes de datos.
- Calcula el número de secuencia del segmento y el número de ACK del segmento de respuesta.

Ejercicio 3:

- **A** envía un segmento a **B** con 300 bytes de datos, el ISN de **A** será 2000.
- ¿Cuál es el número de ACK que envía **B**?
- **A** envía un segundo segmento con 400 bytes
- ¿Cuál es el número de secuencia del segundo segmento?
- Si **B** implementa un ACK acumulado y recibe estos dos segmentos seguidos, con menos de 0.5 segundos de diferencia, ¿qué ocurrirá con los ACK?

Ejercicio para la sesión del 17 de marzo



Ejercicio 4:

- El *host* A, con ISN=3000 y un RTO estimado de 1 segundo, envía un segmento con 250 bytes a B. El segmento se pierde en el camino.
- ¿Qué hace A después de que expire el temporizador de retransmisión?
- ¿Qué número de secuencia y ACK usa en el segmento retransmitido?
- Si este segmento se entrega correctamente, ¿qué número de ACK enviará B?

Ejercicio 5:

- A, con ISN=4000 tras establecer la conexión, envía 6 segmentos consecutivos a B, cada uno con 100 bytes de datos. El segundo y tercer segmento se pierden.
- ¿Qué ocurre con el temporizador de retransmisión en A?
- ¿Qué segmento(s) retransmite A y en qué momento?

Ejercicio 6:

- A, con ISN=5000, envía un segmento con 400 bytes a B y e inicia su TR con RTO=1 seg. El segmento llega correctamente.
- B envía el ACK correspondiente transcurrido su temporizador de 0.5 seg. Este ACK se pierde.
- A envía, antes de que expire su TR, un segundo segmento a B.
- B, que ya ha recibido el primer segmento, envía el ACK esperado una vez transcurrido su propio temporizador.
- El TR de **A** expira antes de recibir este segundo ACK, por lo que se reenvía el primer segmento.
- ¿Qué ocurre con el RTO de **A**? ¿Cómo responde B a esta situación?

Examinar las opciones de TCP



Wireshark

Utiliza Wireshark para capturar el tráfico de tu interfaz de red y, a continuación, establece una conexión TCP, por ejemplo accediendo a una página web, un servidor FTP o de correo.

Filtra los paquetes para buscar aquellos que tienen el bit `SYN` activado, tal y como se aprecia en la imagen de la derecha. El primer segmento de la secuencia contendrá los valores para distintas opciones, como el `MSS`, número de secuencia inicial, etc.

De igual forma puedes buscar segmentos cuyo bit `FIN` esté activo y examinar el intercambio de segmentos que lleva a cerrar una conexión.

ip.proto==6 and tcp.flags.syn==1

No.	Time	Source	Destination	Protocol	Length	Info
3251	11.625916	192.168.2.161	142.250.200.110	TCP	66	6468 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
3269	11.647871	142.250.200.110	192.168.2.161	TCP	66	443 → 6468 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
9459	34.202618	192.168.2.161	1.1.1.1	TCP	66	6469 → 53 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
9460	34.202774	192.168.2.161	1.1.1.1	TCP	66	6470 → 53 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
9463	34.204288	1.1.1.1	192.168.2.161	TCP	66	53 → 6469 [SYN, ACK] Seq=0 Ack=1 Win=42340 Len=0
9464	34.204288	1.1.1.1	192.168.2.161	TCP	66	53 → 6470 [SYN, ACK] Seq=0 Ack=1 Win=42340 Len=0

Transmission Control Protocol, Src Port: 6468, Dst Port: 443, Seq: 0, Win: 64240, Len: 0

Source Port: 6468
Destination Port: 443
[Stream index: 28]
[Conversation completeness: Incomplete, DATA (15)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 1118276748
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1000 = Header Length: 32 bytes (8)
Flags: 0x002 (SYN)
Window: 64240
[Calculated window size: 64240]
Checksum: 0x1ad9 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), Maximum segment size, No-Operation (NOP)
TCP Option - Maximum segment size: 1460 bytes
TCP Option - No-Operation (NOP)

0000 9e 05 d6 49 d6 e0 74 3a f4 88 c5 b6 08 00 45 00
0010 00 34 f0 9c 40 00 80 06 00 00 c0 a8 02 a1 8e fa
0020 c8 6e 19 44 01 bb 42 a7 8c 8c 00 00 00 00 80 02
0030 fa f0 1a d9 00 00 02 04 05 b4 01 03 03 08 01 01
0040 04 02

La utilidad tc

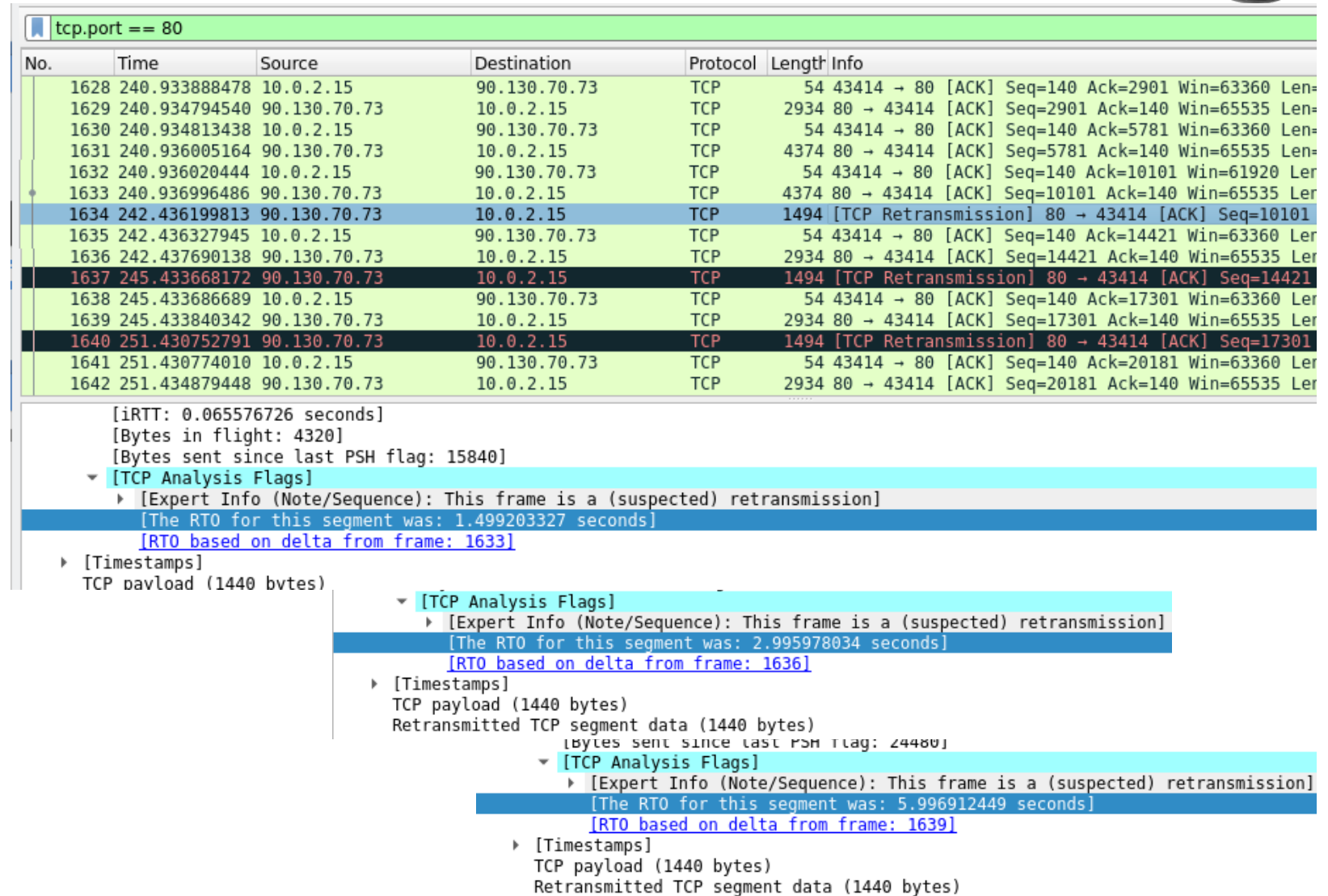
La herramienta `tc` permite alterar distintos parámetros de funcionamiento de una interfaz de red, entre ellas la latencia de envío de paquetes, la pérdida aleatoria de estos, la alteración de su orden, el ancho de banda disponible, etc.

Con la siguiente orden se ajustará, para la interfaz `enp0s3`, la pérdida de paquetes a un 20%:

```
sudo tc qdisc add dev enp0s3 root netem loss 20%
```

Si, tras realizar este ajuste, se envía o recibe un archivo grande a través de TCP, p.e.: `wget http://speedtest.tele2.net/1MB.zip`, en la captura de Wireshark apreciaremos el reenvío de paquetes con el mismo número de secuencia (véase a la derecha).

Al acceder a los detalles de TCP, en el panel intermedio de Wireshark, comprobaremos cómo el valor de RTO se ha duplicado con cada segmento perdido.



The image shows a Wireshark packet capture of a TCP connection. The top pane displays a list of packets, with several retransmissions highlighted in red. The middle pane shows the details of a selected packet (No. 1637), indicating it is a suspected retransmission. The bottom pane shows the details of another selected packet (No. 1640), also indicating it is a suspected retransmission. The RTO (Retransmission Timeout) values are shown for each segment, demonstrating how they increase after a retransmission.

No.	Time	Source	Destination	Protocol	Length	Info
1628	240.933888478	10.0.2.15	90.130.70.73	TCP	54	43414 → 80 [ACK] Seq=140 Ack=2901 Win=63360 Len=
1629	240.934794540	90.130.70.73	10.0.2.15	TCP	2934	80 → 43414 [ACK] Seq=2901 Ack=140 Win=65535 Len=
1630	240.934813438	10.0.2.15	90.130.70.73	TCP	54	43414 → 80 [ACK] Seq=140 Ack=5781 Win=63360 Len=
1631	240.936005164	90.130.70.73	10.0.2.15	TCP	4374	80 → 43414 [ACK] Seq=5781 Ack=140 Win=65535 Len=
1632	240.936020444	10.0.2.15	90.130.70.73	TCP	54	43414 → 80 [ACK] Seq=140 Ack=10101 Win=61920 Len=
1633	240.936996486	90.130.70.73	10.0.2.15	TCP	4374	80 → 43414 [ACK] Seq=10101 Ack=140 Win=65535 Len=
1634	242.436199813	90.130.70.73	10.0.2.15	TCP	1494	[TCP Retransmission] 80 → 43414 [ACK] Seq=10101
1635	242.436327945	10.0.2.15	90.130.70.73	TCP	54	43414 → 80 [ACK] Seq=140 Ack=14421 Win=63360 Len=
1636	242.437690138	90.130.70.73	10.0.2.15	TCP	2934	80 → 43414 [ACK] Seq=14421 Ack=140 Win=65535 Len=
1637	245.433668172	90.130.70.73	10.0.2.15	TCP	1494	[TCP Retransmission] 80 → 43414 [ACK] Seq=14421
1638	245.433686689	10.0.2.15	90.130.70.73	TCP	54	43414 → 80 [ACK] Seq=140 Ack=17301 Win=63360 Len=
1639	245.433840342	90.130.70.73	10.0.2.15	TCP	2934	80 → 43414 [ACK] Seq=17301 Ack=140 Win=65535 Len=
1640	251.430752791	90.130.70.73	10.0.2.15	TCP	1494	[TCP Retransmission] 80 → 43414 [ACK] Seq=17301
1641	251.430774010	10.0.2.15	90.130.70.73	TCP	54	43414 → 80 [ACK] Seq=140 Ack=20181 Win=63360 Len=
1642	251.434879448	90.130.70.73	10.0.2.15	TCP	2934	80 → 43414 [ACK] Seq=20181 Ack=140 Win=65535 Len=

[iRTT: 0.065576726 seconds]
[Bytes in flight: 4320]
[Bytes sent since last PSH flag: 15840]
[TCP Analysis Flags]
[Expert Info (Note/Sequence): This frame is a (suspected) retransmission]
[The RTO for this segment was: 1.499203327 seconds]
[RTO based on delta from frame: 1633]
[Timestamps]
TCP payload (1440 bytes)

[TCP Analysis Flags]
[Expert Info (Note/Sequence): This frame is a (suspected) retransmission]
[The RTO for this segment was: 2.995978034 seconds]
[RTO based on delta from frame: 1636]
[Timestamps]
TCP payload (1440 bytes)
Retransmitted TCP segment data (1440 bytes)
[bytes sent since last PSH flag: 24480]
[TCP Analysis Flags]
[Expert Info (Note/Sequence): This frame is a (suspected) retransmission]
[The RTO for this segment was: 5.996912449 seconds]
[RTO based on delta from frame: 1639]
[Timestamps]
TCP payload (1440 bytes)
Retransmitted TCP segment data (1440 bytes)

Material adicional

Descripción

Para ampliar tus conocimientos sobre los contenidos de esta semana te recomendamos que consultes los recursos indicados a continuación.

Recursos

- **Capítulo 3** La capa de transporte, del libro Redes de computadoras 7ED disponible en [formato digital](#) en la BUJA (recuerda identificarte para poder acceder a leerlo desde tu navegador), concretamente las **secciones 3.1 a la 3.5**
- **TCP/IP Transport Layer Protocols** en el recurso electrónico [The TCP/IP Guide](#), donde encontrarás todos los detalles sobre el funcionamiento de UDP y TCP
- **hping Home** (<http://www.hping.org>) es la [página oficial](#) de la herramienta hping3, en la que encontrarás enlaces a descargas para otros sistemas, documentación, tutoriales, etc.

Cuestiones clave de este tema



Cuestiones clave

Qué deberías saber

*Al inicio de este tema se planteaban unos objetivos específicos que deberían permitirte **responder a las siguientes cuestiones clave***

Cuestiones

- ¿Cómo garantiza TCP la entrega fiable de datos?
- ¿Cuál es la finalidad de los números de secuencia y de reconocimiento?
- ¿Qué método se emplea para controlar el tiempo de retransmisión de segmentos?
- ¿Cómo se reduce el número de segmentos retransmitidos?
- ¿Qué reglas se siguen a la hora de reconocer los segmentos recibidos?